



UNIVERSIDAD
DE GRANADA

Este documento está protegido por la Ley de Propiedad
Intelectual (Real Decreto Ley 1/1996 de 12 de abril).
Queda expresamente prohibido su uso o distribución
sin autorización del autor.

Tecnologías Web

3º Grado en Ingeniería Informática

Guión de prácticas El lenguaje PHP

1. Objetivo.....	2
Sesión 1.....	3
2. Hola mundo.....	4
3. Obtener información externa al script.....	4
4. Calculadora.....	6
5. Formularios y validación en el servidor.....	8
Sesión 2.....	12
6. Establecer preferencias en un sitio web.....	13
7. Almacenamiento de datos en variables de sesión.....	15
8. Problema con reenvío de formularios.....	16
9. Inyección de código y saneamiento de datos.....	18
10. Tres en raya.....	19
Sesión 3.....	22
11. Instalación y configuración de un servidor MySQL.....	23
12. Creación de una BBDD.....	24
13. Edición de usuarios.....	26
14. Listado paginado.....	26
15. Entrega de la práctica.....	27

© Prof. Javier Martínez Baena
Dpto. Ciencias de la Computación e I. A.
Universidad de Granada



Departamento de
Ciencias de la Computación
e Inteligencia Artificial

1. Objetivo

El objetivo que se plantea es practicar con diferentes aspectos necesarios para el desarrollo de cualquier aplicación web tales como:

- Procesado de datos de formularios.
- Validación de formularios.
- Algunas cuestiones relativas a la seguridad de las aplicaciones web.
- Uso de cookies.
- Uso de variables de sesión.
- Acceso a bases de datos.

Se establecen tres sesiones o bloques temáticos para realizar el guión.

Sesión 1

Introducción a PHP

En esta primera parte del guión se introduce la programación con PHP en un entorno web. Como elemento básico se establece la manipulación de datos a partir de un formulario web.

2. Hola mundo

► Ejercicio 1: Hola mundo en consola

Para comprobar que la instalación de PHP en su ordenador es correcta cree un fichero `holamundo.php` y escriba en él el siguiente programa:

```
<?php
print("Hola mundo".PHP_EOL)
?>
```

Puede ejecutar el código invocando al intérprete desde línea de órdenes con la instrucción:

```
php -f holamundo.php
```

Para ejecutar este mismo programa a través de un navegador debe tener instalado y configurado Apache de manera que este sepa cómo invocar al intérprete. Si es el caso, ubique el fichero `holamundo.php` dentro de su carpeta `public_html` y escriba la URL <https://localhost/~USUARIO/holamundo.php>. Si, desde el navegador, mira el código fuente de la página verá que el contenido es exactamente la cadena "Hola mundo".

► Ejercicio 2: Hola mundo en la web

En el ejercicio anterior el resultado de la ejecución del *script* es texto plano por lo que no sería un resultado adecuado para visualizarlo en un navegador. Para hacer que su programa PHP genere código HTML válido debe escribir la página web completa usando instrucciones como `print` o `echo`:

Cadena normal	Cadena de tipo heredoc
<pre><?php print("<!DOCTYPE html> <html lang=\"es\"> <head> <meta charset=\"UTF-8\"> <title>Hola mundo</title> </head> <body> <h1>Hola mundo</h1> </body> </html>") ?></pre>	<pre><?php echo <<<HTML <!DOCTYPE html> <html lang="es"> <head> <meta charset="UTF-8"> <title>Hola mundo</title> </head> <body> <h1>Hola mundo</h1> </body> </html> HTML; ?></pre>

3. Obtener información externa al script

Cuando comienza la ejecución de un *script* PHP, el servidor web pone a su disposición alguna información sobre la petición que se está procesando. Una parte de esta se encuentra en una variable global llamada `$_SERVER` que crea el servidor tras recibir la petición HTTP y que existe desde el comienzo de la ejecución del *script*. Entre otras cosas dispone de:

- Información recibida en la petición HTTP:
 - Agente de usuario que hace la petición (`HTTP_USER_AGENT`).
 - *Host* que hace la petición (`HTTP_HOST`).
 - Otras posibles cabeceras como: `HTTP_ACCEPT`, `HTTP_ACCEPT_ENCODING`, `HTTP_CONNECTION`, etc.
- Información sobre el servidor web que está ejecutando el script:
 - De qué servidor se trata (`SERVER_SOFTWARE`, `SERVER_SIGNATURE`).
 - Algunas otras cosas sobre la configuración del servidor (`DOCUMENT_ROOT`, `SERVER_PROTOCOL`, etc.).

Además, `$_SERVER` contiene otra información que es de uso muy frecuente y que tiene que ver con la URI solicitada por el usuario y el *script* que se está ejecutando:

- **REQUEST_URI**: la URL que solicitó el usuario (sin el *host*). Observación: el fragmento de la petición no llega nunca al servidor (si queremos que se envíe debemos hacerlo desde el cliente con JavaScript y añadir esa información en alguna cabecera del mensaje HTTP).
- **QUERY_STRING**: la parte de la URI con los datos adjunto (si los hay).
- **SCRIPT_FILENAME**: la ruta absoluta del *script* que se está ejecutando actualmente.
- **SCRIPT_NAME**: la ruta del *script* que se está ejecutando actualmente.
- **PHP_SELF**: El nombre del *script* que se está ejecutando actualmente (relativa al directorio raíz de documentos del servidor).

Observe que la información contenida en `$_SERVER` puede variar entre distintos servidores (Apache, NGINX, etc.) por lo que el conocimiento sobre el servidor web que estamos usando puede ser importante en este punto.

► Ejercicio 3: Mostrar la información contenida en `$_SERVER`

Implemente un *script* PHP que muestre la información recibida en la variable superglobal `$_SERVER` y compruebe si están los datos que se han comentado.

Suponiendo que ha hecho este ejercicio en un documento llamado `ver_informacion.php` y que este está alojado en la carpeta `practicaPHP` de su `public_html` de `void.ugr.es`, pruebe a hacer las siguientes peticiones:

- https://void.ugr.es/~USUARIO/practicaPHP/ver_informacion.php
- https://void.ugr.es/~USUARIO/practicaPHP/ver_informacion.php?nombre=pepito
- https://void.ugr.es/~USUARIO/practicaPHP/ver_informacion.php#fragmentoquenollegara

Observe y analice el contenido de la información aportada por `$_SERVER`.

3.1. Diferencias entre `SCRIPT_NAME` y `PHP_SELF`

En algunas aplicaciones web se suele incluir en la propia URL información adicional para determinar qué es lo que quiere hacer el usuario. Una forma de hacerlo es usar una sintaxis como esta:

https://void.ugr.es/~USUARIO/practicaPHP/ver_informacion.php/sigue/por/aqui

Puede ver que en lugar de usar un *query string* para enviar información se usa, tras el nombre del *script*, una sintaxis similar a la de una ruta (*trailing path*). Puede ver que esa información adicional está contenida en `$_SERVER['PHP_SELF']` pero no en `$_SERVER['SCRIPT_NAME']` (u otras).

► Ejercicio 4: Usando el *query string*

Realice un *script* que muestre distintos contenidos dependiendo del *path* que sigue al nombre del *script*. Concretamente, el *script* (al que llamaremos *operar.php*) realizará distintas operaciones aritméticas y mostrará el resultado atendiendo peticiones como estas:

- <https://void.ugr.es/~USUARIO/practicaPHP/operar.php/multiplica?a=2&b=3>
- <https://void.ugr.es/~USUARIO/practicaPHP/operar.php/suma?a=1&b=6>
- <https://void.ugr.es/~USUARIO/practicaPHP/operar.php/divide?a=4&b=8>

Para ello deberá analizar la variable y extraer la parte que se corresponde con el *trailing path*. Se puede hacer de varias formas y una de ellas es mediante expresiones regulares combinado con alguna función de manipulación de cadenas. A continuación tiene un ejemplo:

```
preg_match('#([^\/*]*php)/(.*)$#', $_SERVER['PHP_SELF'], $coincidencias);
$chunks = (count($coincidencias)>0) ? explode('/', $coincidencias[2]) : [];
if (count($chunks)>0)
    echo '<pre>' . var_export($chunks, true) . '</pre>';
else
```

```
echo'<p>No hay trailing path</p>';
```

En un caso como el expuesto en este ejercicio se necesita hacer uso de `PHP_SELF` porque necesitamos utilizar la parte de la URI que va tras el nombre del *script*. Si no necesita esa información extra seguramente bastará con que use `SCRIPT_NAME`.

Un uso inadecuado de `PHP_SELF` podría suponer un riesgo de seguridad. Considere el siguiente código HTML/PHP:

```
<form method="post" action="<?php echo $_SERVER['PHP_SELF']; ?>">
  <!-- form contents -->
</form>
```

Si incluye ese formulario en un script PHP llamado, por ejemplo, `pruebaform.php` y a continuación solicita la siguiente URL:

```
https://void.ugr.es/.../pruebaform.php/"><script>alert(1)</script>
```

Podrá ver que de forma colateral está ejecutando el código *JavaScript* que se ha inyectado a través de la URL.

3.2. Diferencias entre la URI y el script que se está ejecutando

► Ejercicio 5: Más datos contenidos en `$_SERVER`

Este ejercicio plantea una situación en la que se aprecia la diferencia entre las variables que contienen el nombre del *script* en ejecución (`SCRIPT_FILENAME` y otras) y la URI solicitada (`REQUEST_URI`), que no tienen porqué coincidir. Para realizarlo vamos a suponer que tenemos el siguiente *script*, llamado `procesarerror.php`, en la carpeta `practicaPHP` dentro de `public_html`:

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8" />
<title>Ejemplo</title>
</head>
<body>
<?php
echo '<p>Si has llegado aquí es porque la URL que has puesto es incorrecta</p>';
echo '<pre>' . var_export($_SERVER, true) . '</pre>';
?>
</body>
</html>
```

A continuación, incluya un fichero `.htaccess` en esa carpeta con el siguiente contenido:

```
RewriteEngine on
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule . procesarerror.php [L]
```

Con ese fichero se le está indicando a Apache que, en caso de recibir una petición de un recurso en la carpeta `~USUARIO/practicaPHP` que ni es un fichero ni es un directorio (es decir: que no existe), reescriba la URI sustituyendo dicho recurso por `procesarerror.php`.

Pruebe a solicitar la siguiente URL:

```
https://void.ugr.es/~USUARIO/practicaPHP/unadireccionquenoexiste.php
```

Compruebe que se ha ejecutado su script `procesarerror.php` y analice el contenido de las variables indicadas al comienzo de este ejercicio.

4. Calculadora

► Ejercicio 6: Calculadora

En este ejercicio debe implementar un programa calculadora en PHP. Esta permitirá hacer las cuatro operaciones aritméticas básicas (suma, resta, multiplicación y división) sobre dos argumentos numéricos. A modo de ejemplo, a continuación puede ver varias capturas de pantalla que muestran el comportamiento de la aplicación ante diversas situaciones:

The screenshots show a web calculator interface with the following components: a title 'Calculadora', two input fields 'Dato 1' and 'Dato 2', a central 'Operación' button with four options (+, -, *, /), and a 'Resultado' field. The interface is light blue with white text and buttons.

- Screenshot 1:** Initial state. 'Dato 1' and 'Dato 2' are empty. 'Operación' is set to '+'. 'Resultado' is empty.
- Screenshot 2:** Successful calculation. 'Dato 1' contains '23', 'Dato 2' contains '56', 'Operación' is set to '*', and 'Resultado' displays '1288'.
- Screenshot 3:** Error state. 'Dato 1' is empty, 'Dato 2' contains 'Esto no es un número'. Below the inputs, two red error messages are displayed: 'ERROR: El primer dato no es válido' and 'ERROR: El segundo dato no es válido'.
- Screenshot 4:** Error state. 'Dato 1' contains '4', 'Dato 2' contains '56', and 'Operación' is set to '/'. Below the inputs, a red error message is displayed: 'ERROR: El primer dato no es válido'.
- Screenshot 5:** Error state. 'Dato 1' contains '4', 'Dato 2' contains '0', and 'Operación' is set to '/'. Below the inputs, a red error message is displayed: 'ERROR: División por cero'.

Observe que la aplicación debe hacer comprobaciones sobre la validez de los argumentos antes de realizar la operación:

- Si los campos de entrada están vacíos.
- Si los campos contienen valores numéricos.
- Si la operación está definida (división por cero).

En caso de que haya algún valor incorrecto, se presentará de nuevo la interfaz de la calculadora informando de la causa del error. Si alguno de los argumentos es válido se conservará su valor en el formulario.

En <https://void.ugr.es/practicas/calculadora.php> puede probar el *script*, que está basado en el siguiente código CSS+HTML:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Calculadora</title>
  <style>
    main {
      font-family: Arial;
```

```

        display: flex;
        flex-direction: column;
        align-items: center;
    }
    form {
        border: 2px solid lightgray;
        padding: 5px;
        display: inline-flex;
        align-items: center;
        background-color: lightblue;
    }
    fieldset {
        display: flex;
        flex-direction: column;
    }
    label {
        margin: 10px;
        display: flex;
        flex-direction: column;
    }
    .error {
        color: red;
    }
</style>
</head>
<body>
    <main>
        <h1>Calculadora</h1>
        <form action="" method="GET">
            <label><span>Dato 1</span><input type="text" name="numero1" placeholder="Introduce un
número"/></label>
            <fieldset>
                <legend>Operación</legend>
                <input type="submit" name="suma" value="+">
                <input type="submit" name="resta" value="-">
                <input type="submit" name="producto" value="*">
                <input type="submit" name="division" value="/">
            </fieldset>
            <label><span>Dato 2</span><input type="text" name="numero2" placeholder="Introduce un
número"/></label>
        </form>
        <section>
            <p>Operación: <span>Sin operación</span></p>
            <p>Resultado: <span>Sin resultado</span></p>
        </section>
    </main>
</body>
</html>

```

5. Formularios y validación en el servidor

En prácticas anteriores diseñó un formulario de recogida de datos de los clientes de un hotel.

► Ejercicio 7: Validación de formularios

Implemente un *script* PHP para trabajar con dicho formulario. El ejercicio se realizará usando un único fichero PHP en el que se incluirá todo lo necesario. Cuando se solicite este fichero desde un cliente el resultado podrá variar dependiendo de algunos factores:

- Se solicita el *script* por primera vez. O, dicho de otra forma, se solicita el *script* sin haber pulsado el control de **submit** del formulario. En este caso se muestra el formulario sin

rellenar.

- Se rellena el formulario y se pulsa el botón de envío. En este caso tenemos dos opciones:
 - Uno o más datos son incorrectos. Se mostrará de nuevo el formulario con las siguientes condiciones:
 - Los datos que sean correctos serán incluidos en sus correspondientes controles de entrada para que el usuario no tenga que volver a escribirlos (*sticky form*).
 - Los datos incorrectos no se incluirán pero sí se avisará junto al control del tipo de error encontrado.
 - Los datos son todos correctos. Se muestran los datos recibidos reutilizando el mismo formulario pero deshabilitando los controles de entrada.

La siguiente figura muestra el formulario vacío cuando se solicita por primera vez (izquierda) y el formulario que muestra el *script* si pulsamos el botón de enviar datos sobre el formulario vacío (derecha). Puede ver que junto a aquellos campos en los que se detecta algún error se muestra un mensaje informativo en rojo.

En la siguiente figura puede ver el resultado de rellenar algunos campos bien y otros mal (izquierda). El formulario conserva los valores correctos y borra los campos erróneos. En la figura central se han rellenado algunos campos (fecha de nacimiento y email) con valores incorrectos. Tras pulsar sobre el botón de envío se ve el resultado (derecha) con mensajes de error adaptados a las circunstancias:

Para validar correctamente los datos debe cumplirse lo siguiente:

- El nombre no puede estar vacío.
- No es obligatorio indicar los apellidos.
- El DNI no puede estar vacío y la letra debe ser correcta.
- La fecha de nacimiento no puede estar vacía y la persona debe ser mayor de edad.
- No es obligatorio indicar la nacionalidad.
- El sexo debe ser uno de los tres valores permitidos.
- El email no puede estar vacío y debe ser una dirección válida. Utilice `filter_var()` y `FILTER_VALIDATE_EMAIL`.

- La clave no puede estar vacía y han de coincidir ambas claves.
- El idioma no puede estar vacío y debe ser uno de los tres valores permitidos.
- No es obligatorio marcar preferencias.

Aunque en las figuras previas se ha incluido un control de tipo **file** para subir la fotografía del usuario en este ejercicio deberá omitirlo o, simplemente, ignorarlo. El formulario debe enviarse obligatoriamente mediante el método **GET** para poder ver los datos enviados en la URL. Al hacer el envío con **GET** no se subirá ningún fichero al servidor (aunque verá en la URL el nombre del fichero seleccionado).

Observe que aunque algunos controles parece que es imposible que tengan valores incorrectos (**select**, **radio**, **checkbox**, ...) también deben ser validados comprobando que tienen valor y que es alguno de los que se esperan. Recuerde que cualquier dato que venga de un cliente externo puede ser falseado. Si está enviando el formulario con **GET** puede modificar la URL para omitir algún parámetro o cambiar su valor a cualquier cosa. Si se envía con **POST** también se puede hacer aunque para ello necesitaría herramientas adicionales al navegador (por ejemplo **cURL**).

Cuando se reciban todos los datos y sean validados correctamente se mostrará una información como la siguiente:

Los datos se han recibido correctamente

Registro de usuarios

Datos personales

Nombre DNI
 Apellidos F. nacimiento
 Nacionalidad Fotografía Ningún archivo seleccionado
 Sexo

Datos de acceso

Email
 Clave
 (repítala)

Datos adicionales 1

Idioma para comunicaciones:
☐ Español
☐ Inglés
☐ Francés

Datos adicionales 2

Aficiones:
☐ La música
☒ Leer libros
☒ El cine
☐ El deporte

Tratamiento de datos

Puede ver que hay un mensaje informativo al comienzo “Los datos se han recibido correctamente” y que la información recibida se muestra con el mismo formulario pero esta vez de forma que los campos no son editables para evitar que sean modificados de nuevo. Además, el texto que aparece sobre el botón ha cambiado a “Confirmar datos”. Esto es algo frecuente en los procesos de envío de información a un servidor: una vez recibidos y validados los datos, estos se le presentan al usuario para que pueda revisar que todo es correcto antes de procesarlos de manera definitiva (y, por ejemplo, almacenarlos en una base de datos). Para esto último no es estrictamente necesario el formulario y se podría haber escrito el valor de cada dato, sin embargo, presentar los datos con el mismo formato de entrada (el formulario) hará que el usuario pueda revisarlos de forma más cómoda.

Para hacer que los controles de entrada no sean modificables dispone de dos atributos:

- **disabled**. Deshabilita la entrada de datos en un control y este se muestra con un fondo grisáceo. El inconveniente de este atributo es que los campos marcados con él no son enviados al pulsar el botón de **submit**.
- **readonly**. Deshabilita la entrada de datos en un control. En este caso los campos sí se envían al hacer **submit**. El inconveniente es que no se puede utilizar con controles de tipo **select**, **radio** o **checkbox** por lo que nos obliga a buscar alguna solución que permita mostrar el control sin posibilidad de modificarlo y, a la vez, que envíe la información al hacer **submit**. Recuerde que en este ejercicio no se permite el uso de código JavaScript.

En relación con la validación de datos, debe notar que algunos tipos de entrada (**date**, **email**, ...) ya son validados por el navegador antes de su envío al servidor. Esto hace que el proceso de introducción de datos sea más ágil para el usuario (se evita el envío de datos que se sabe que son incorrectos). Sin embargo, es algo fácilmente sorteable por lo que siempre será obligatorio validarlos en el servidor. Para evitar la validación en el cliente y poder comprobar que la validación en el servidor está funcionando debe utilizar el atributo **novalidate** del formulario.

Llegados a este punto puede ver que se va a mostrar el formulario con tres variantes:

- Formulario en blanco (inicial).
- Formulario *sticky*. Se presenta el mismo formulario con algunos controles que tienen valores ya preestablecidos (**value**) y con posibles avisos de error.
- Formulario con controles rellenos y deshabilitados.

Debe evitar reescribir código para las distintas situaciones (copia/pega o variantes). Construya una única función para presentar el formulario y use parámetros para disponer de todas las variantes.

En este ejercicio no podrá hacer uso de *JavaScript* para la validación en el cliente.

Sesión 2

Estableciendo conexiones “*statefull*” Saneado de datos

Una de las características del protocolo HTTP es que es *stateless*, es decir, que no implementa ningún mecanismo para recordar el estado de una conexión. Esto implica que si simplemente nos basamos en la definición del protocolo, no es posible recordar el historial de actividad de un cliente en una aplicación web alojada en un servidor. Para dar solución a esta situación se aborda el uso de las cookies y las sesiones.

Observe que esto no cambia el hecho de que el protocolo sea *stateless*. Simplemente es un mecanismo que, haciendo uso del protocolo HTTP, permite recordar el estado de las aplicaciones.

También se incluye algún ejercicio para hacer el saneado de datos de formularios.

6. Establecer preferencias en un sitio web

► Ejercicio 8: Uso de cookies

Una situación que ocurre con frecuencia en sitios web se da cuando el usuario quiere hacer algún tipo de personalización como, por ejemplo, idioma, colores, etc. Esto se puede resolver de diferentes formas pero una muy simple pasa por hacer uso de las *cookies*.

En este ejercicio debe crear un *script* que muestre el contenido de una web adaptado al idioma elegido por el usuario de forma que, una vez seleccionado el idioma, todas las páginas web del sitio se muestran acorde a él. La selección del usuario se va a mantener en futuras visitas al sitio.

Para este ejercicio debe crear un único *script* que muestre, obligatoriamente:

- Un mensaje de bienvenida.
- Un formulario para elegir idioma.
- Un *link* para cargar de nuevo la página. También podría ser para ver otras páginas del sitio si las hubiera.

La página podrá visualizarse en tres idiomas distintos: español, inglés y francés. Puede ver un ejemplo de la página en los distintos idiomas a continuación:

The figure shows three overlapping screenshots of a web page in different languages: Spanish, English, and French. Each screenshot displays a welcome message, a language selection form with a dropdown menu and an 'Apply' button, and a 'Reload this page' link.

- Spanish (top-left):** "Hola, bienvenido a la web. Usted está viendo esta información en castellano." "Puede cambiar el idioma en el formulario adjunto." "Elija idioma" (dropdown: Español) "Aplicar cambios" "Cargar de nuevo esta página"
- English (bottom-left):** "Hello, welcome to the web. You english." "You can change the language in the attached form." "Choose language" (dropdown: English) "Apply changes" "Reload this page again"
- French (top-right):** "Bonjour, bienvenue sur le web. Vous consultez ces informations en français." "Vous pouvez changer la langue dans le formulaire ci-joint." "Choisissez la langue" (dropdown: Français) "Appliquer les modifications" "Recharger cette page"

Para facilitar el mantenimiento del software, almacenaremos los mensajes de texto de cada idioma en un fichero con formato JSON. También podríamos optar por almacenar cada idioma en un fichero distinto. A continuación tiene el fichero con los mensajes del ejemplo de la figura:

```
{
  "es" : {
    "ElegirIdioma" : "Elija idioma",
    "Espanol" : "Español",
    "Ingles" : "Inglés",
    "Frances" : "Francés",
    "Bienvenida" : "Hola, bienvenido a la web. Usted está viendo esta información en castellano.",
    "Cambio" : "Puede cambiar el idioma en el formulario adjunto.",
    "Aplicar" : "Aplicar cambios",
    "Enlace" : "Cargar de nuevo esta página"
  },
  "en" : {
    "ElegirIdioma" : "Choose language",
    "Espanol" : "Spanish",
```

```

        "Ingles" : "English",
        "Frances" : "French",
        "Bienvenida" : "Hello, welcome to the web. You are viewing this information in
        english.",
        "Cambio" : "You can change the language in the attached form.",
        "Aplicar" : "Apply changes",
        "Enlace" : "Reload this page again"
    },
    "fr" : {
        "ElegirIdioma" : "Choisissez la langue",
        "Espanol" : "Espagnol",
        "Ingles" : "Anglais",
        "Frances" : "Français",
        "Bienvenida" : "Bonjour, bienvenue sur le web. Vous consultez ces informations
        en français.",
        "Cambio" : "Vous pouvez changer la langue dans le formulaire ci-joint.",
        "Aplicar" : "Appliquer les modifications",
        "Enlace" : "Recharger cette page"
    }
}

```

Para cargar un documento JSON desde PHP puede usar este código:

```
$mensajes = json_decode(file_get_contents('mensajes.json'), true);
```

Si hay algún error en la lectura o sintaxis del documento la función devuelve `null`.

Lo primero que debe hacer el *script* cuando se ejecute es determinar el idioma con el que se debe mostrar la página y para ello puede ocurrir que:

- El usuario ha usado el formulario y ha pulsado el botón de aplicar modificaciones. En este caso el idioma viene determinado por la selección del formulario. Además, se debe establecer una *cookie* para recordar la selección en futuras visitas.
- El usuario está visitando la página pero no porque haya usado el formulario. La situación típica es que el usuario navegue por todas las páginas de nuestro sitio una vez que ya estableció el idioma con anterioridad. En este ejercicio solo debe desarrollar una página pero podría haber más. En este caso puede ocurrir que:
 - El usuario ya estableció una preferencia con anterioridad mediante el formulario. En este caso dicha preferencia estará almacenada en una *cookie*.
 - El usuario no estableció ninguna preferencia previamente. En este caso se mostrará la página en el lenguaje por defecto (Español).

Una vez determinado el idioma se creará el documento HTML usando los mensajes dependiendo del idioma. Observe también que el formulario mantiene como opción por defecto la que se corresponde con el idioma actual.

Cuando tenga el *script* en funcionamiento:

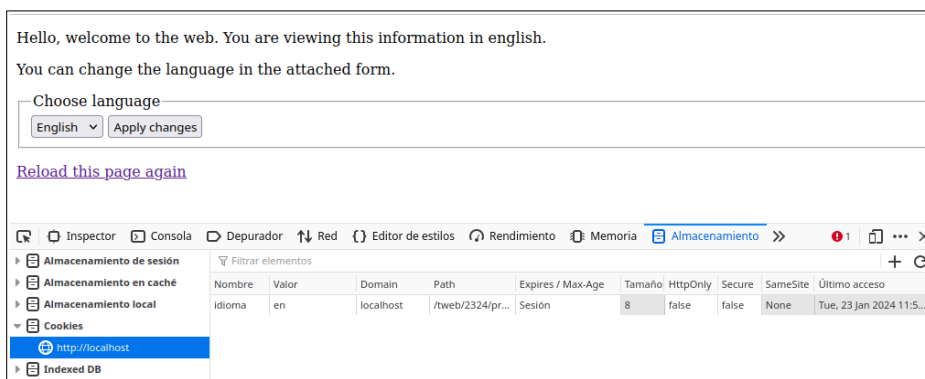
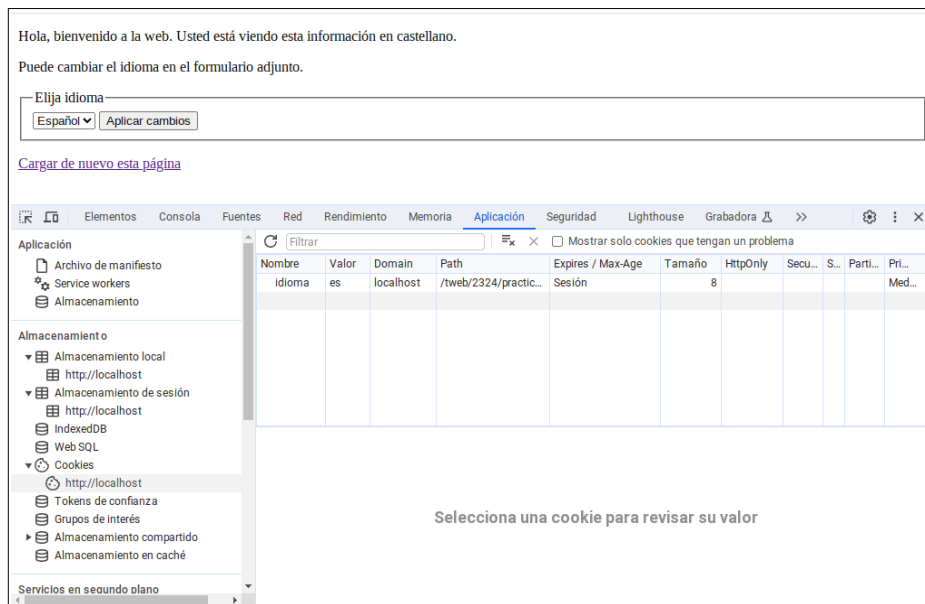
- Utilice las herramientas de desarrollo del navegador para ver cómo se envía la *cookie* en el mensaje HTTP.
- Modifique la *cookie* desde su navegador y cambie su valor. El efecto debería ser el mismo que si la hubiese cambiado desde el formulario. Puede comprobar que el uso de las cookies es inseguro por naturaleza al ser modificables desde el entorno del cliente con facilidad.

Nota importante: si cuando pulsa el botón de “Aplicar cambios” no surte efecto el cambio de idioma y debe pulsarlo de nuevo o recargar la página para que cambie el idioma entonces no está haciendo bien el ejercicio. Para que el ejercicio sea correcto el cambio de idioma debe producirse al pulsar el botón. No puede hacer redirecciones para resolver el ejercicio.

► Ejercicio 9: Edición de cookies en el navegador

Cuando haya hecho el ejercicio anterior pruebe a modificar el valor de la *cookie* desde las herramientas de desarrollo del navegador. Dependiendo del navegador la ubicación de esta

herramienta puede cambiar. A continuación ve dos figuras: una para Chrome y otra para Firefox.



Para ver que esta modificación surte efecto, tras cambiar el valor de la *cookie* pulse el enlace para cargar la página de nuevo (no pulse el botón de aplicar cambios).

7. Almacenamiento de datos en variables de sesión

► Ejercicio 10: Sesiones de usuario

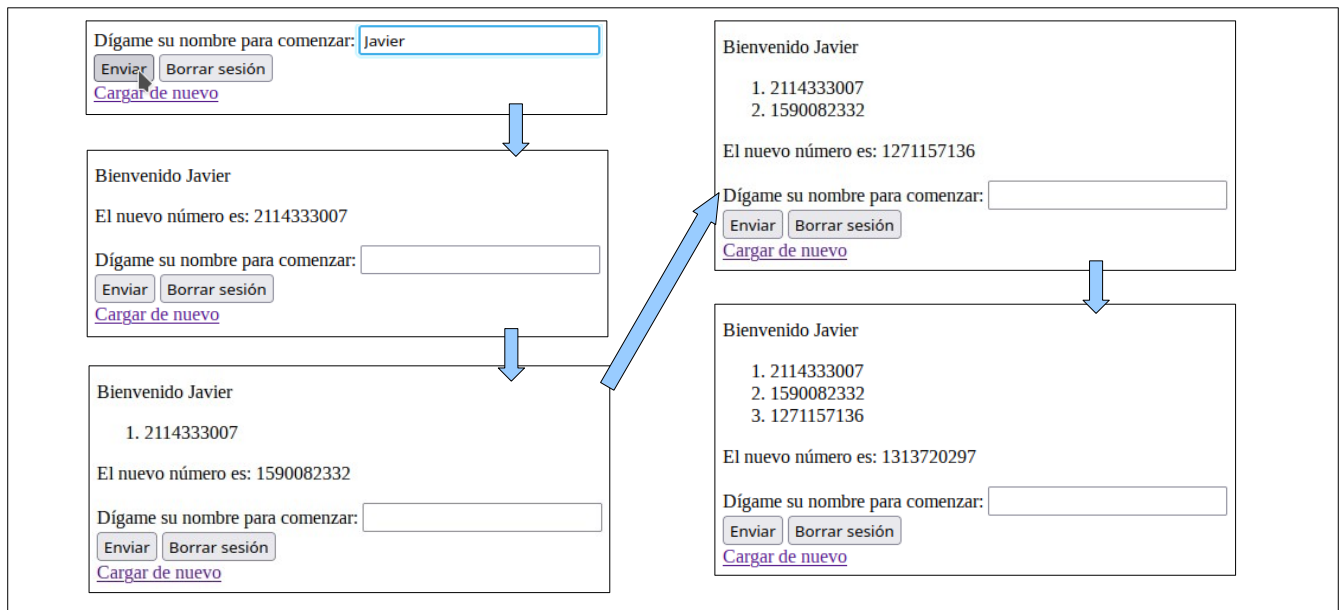
El uso de *cookies* tiene algunas limitaciones a tener en cuenta. En particular:

- La cantidad de información que se puede almacenar es limitada.
- La información se almacena en el cliente por lo que puede plantear problemas de seguridad.

Desarrolle un *script* que, la primera vez que un usuario visite la página le pregunte su nombre. Al aceptar y enviar el formulario le aparecerá una nueva página en donde se le dé la bienvenida con su nombre y añada un número aleatorio a continuación. En cada nueva visita a la página se mostrará el mensaje de bienvenida seguido de una lista con todos los números aleatorios que se han generado hasta el momento seguidos de otro nuevo.

Toda la información (nombre y números) será almacenada en variables de sesión.

A modo de ejemplo, a continuación tiene varias capturas de pantalla. En la primera visita rellenamos el nombre y a partir de la segunda vemos el resultado de recargar la página (bien con el botón de recargar del navegador o bien pulsando el enlace “Cargar de nuevo”):



Si el usuario rellena de nuevo un nombre y pulsa “Enviar”, el resultado es que se reinicia la lista y comienza de nuevo el proceso generando un primer número aleatorio.

Si el usuario pulsa “Borrar sesión” el resultado es que se reinicia todo y se muestra el primer formulario (el efecto es como si visitásemos la página por primera vez).

8. Problema con reenvío de formularios

► Ejercicio 11: El problema de “double submit”

Copie el siguiente *script* en un fichero. El código simula el envío de donaciones a una ONG:

```

<?php
session_start();

if (isset($_POST['borrar']))
    unset($_SESSION['total']);

if (!isset($_SESSION['total']) or !isset($_SESSION['numero'])) {
    $_SESSION['numero'] = 0;
    $_SESSION['total'] = 0;
}

if (isset($_POST['enviar']) and is_numeric($_POST['donacion'])) {
    $_SESSION['numero'] ++;
    $_SESSION['total'] += $_POST['donacion'];
}

?>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8" />
    <title>Ejemplo</title>
</head>
<body>
    <?php
    echo "<p>Hasta ahora hay un total de {$_SESSION['numero']} donaciones.</p>";
    echo "<p>El importe acumulado es de {$_SESSION['total']} euros.</p>";
    if (isset($_POST['donacion']) and is_numeric($_POST['donacion']))
        echo "<p>La última donación fue de {$_POST['donacion']} euros</p>";
  
```

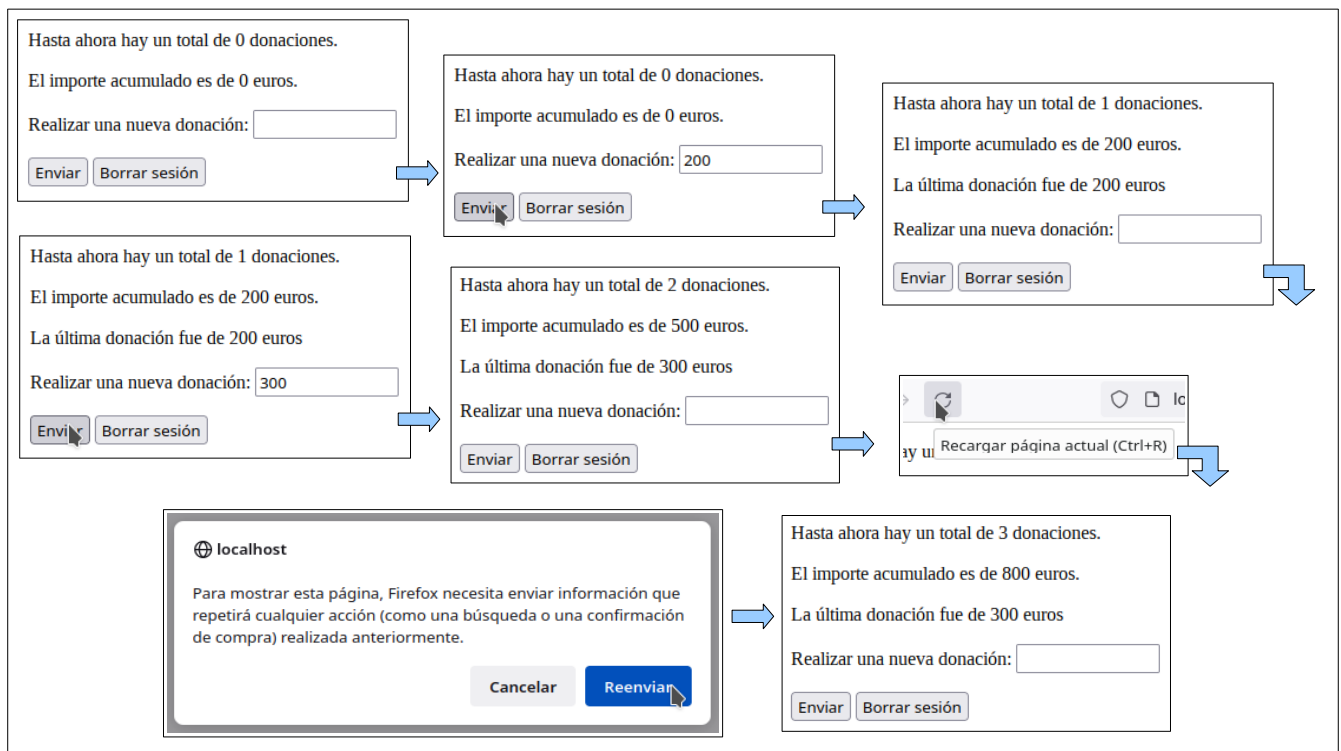


```

    echo form();
    ?>
</body>
</html>
<?php
function form() {
    return <<<HTML
        <form action="{$_SERVER['SCRIPT_NAME']}" method="post">
            <label>Realizar una nueva donación: <input type="text" name="donacion"
size="10"></label>
            <p><input type="submit" name="enviar" value="Enviar">
                <input type="submit" name="borrar" value="Borrar sesión"></p>
        </form>\n
    HTML;
}
?>

```

La página muestra el total recaudado hasta el momento, el número de donaciones realizadas y la cantidad donada en la última visita a la página. La figura siguiente muestra una secuencia de operaciones sobre la página para hacer algunas donaciones:



Puede ver que tras la donación de 300 euros, se ha pulsado el botón de recargar la página del navegador. Al hacer esto el navegador nos muestra una advertencia para que seamos conscientes de que al hacer la recarga, lo que estamos haciendo es repetir la última petición **POST** al servidor y que eso implica volver a enviar los datos de la última petición también. Si aceptamos vemos que, aun no habiendo rellenado el formulario, se ha vuelto a contabilizar la última donación que se hizo.

Este problema se conoce como *double submit problem* y ocurre cuando hacemos recargas o pulsamos hacia atrás en el historial de navegación estando en páginas que han enviado datos en un **POST**. Los navegadores, siendo conscientes del riesgo de reenviar los mismos datos más de una vez, advierten del problema. Sin embargo, no es una buena solución ya que no evitan que ocurra sino que únicamente advierten de que va a ocurrir si continuamos la acción. Además, este tipo de mensajes pueden ser confusos o poco informativos para usuarios de nuestra aplicación sin conocimientos técnicos.

► Ejercicio 12: Solución del problema de "double submit"

Por lo tanto, deberíamos resolver esto desde nuestra propia aplicación. Existen diversas aproximaciones para su resolución. Una de ellas muy extendida es la que se conoce como patrón PRG (POST/REDIRECT/GET)¹². Consiste en que, cuando el servidor recibe una petición **POST**, lo que devuelve no es el resultado del procesamiento de los datos sino un mensaje HTTP con un redireccionamiento a la misma página pero con **GET**, de forma que se eliminen los datos del cuerpo. Recuerde que las redirecciones las hace el navegador (y no el servidor) por lo que la secuencia de mensajes HTTP sería esta:

1. Cliente: hace una petición **POST** y adjunta datos del formulario en el cuerpo del mensaje.
2. Servidor: recibe la petición y la procesa (usa los datos del formulario).
3. Servidor: si el procesamiento de los datos ha tenido éxito lo que hace es enviar un **header** para redireccionar a la misma URL con un código 303:

```
header( "Location: {$_SERVER['SCRIPT_NAME']}", true, 303);
```
4. Cliente: recibe respuesta con **header** 303 y hace una petición **GET** a la URL indicada. Ya no se adjuntan datos del formulario.
5. Servidor: recibe una petición **GET** y devuelve una respuesta.

Observe que si la respuesta que debe mostrar nuestra aplicación en el punto 5 necesitase hacer uso de datos enviados desde el formulario rellenado en el punto 1, estos ya no estarán accesibles en `$_POST` puesto que al punto 5 se llega mediante una petición **GET**. Por tanto, para resolver esto sería necesario que en el punto 2 se guardasen en variables de sesión aquellos datos recibidos en `$_POST` que pudiesen hacer falta en el punto 5.

En este ejercicio debe arreglar el *script* proporcionado para evitar el problema del doble envío manteniendo exactamente la misma funcionalidad. Observe que en la respuesta se muestra la cantidad de la última donación, que es un dato que se envía desde el formulario inicial (punto 1) y que se muestra en la respuesta (punto 5).

9. Inyección de código y saneamiento de datos

Recupere el ejercicio de validación de datos del formulario de registro de clientes de un hotel y realice el siguiente ejercicio a partir de él. Debe crear un nuevo *script* para este ejercicio (y mantener el anterior intacto).

► Ejercicio 13: Inyección HTML y saneamiento de datos

En este ejercicio debe añadir un mensaje de texto de bienvenida como el que ve en la figura solo en el caso de que se haya escrito algo en el campo del nombre.

En el mensaje informativo se incluyó el nombre y el código PHP/HTML podría ser similar a este:

```
<p>Bienvenido/a {$_POST['nombre']}, siga instrucciones a continuación:</p>
```

Pruebe a rellenar el formulario introduciendo en el nombre alguno de los siguientes datos:

- `<h1>Alberto</h1>`
- `<h1>Alberto`
- `<script>alert('Hola, soy JavaScript');</script>`
- `<script>alert(document.cookie);</script>`

1 <https://en.wikipedia.org/wiki/Post/Redirect/Get>

2 <https://icodemag.com/prg-pattern-in-php-what-why-and-how/>

En los cuatro casos se ha producido una inyección de código (HTML en los dos primeros y *JavaScript* en los otros dos) que afecta a la seguridad de nuestra aplicación. Con la inyección HTML podemos romper el maquetado de un sitio web y con la inyección de código *JavaScript* podemos tener un grave problema de seguridad. En el último ejemplo estamos mostrando las *cookies* que ha recibido nuestro *script* desde el cliente con información sensible o con información que permite que otros usuarios se hagan pasar por nosotros. En este caso simplemente mostramos la *cookie* en una ventana pero se podría introducir código para enviar la *cookie* u otra información a un servidor de un tercero con intenciones desconocidas y sin que seamos conscientes de ello.

Por ello, se considera fundamental realizar un buen saneamiento de datos antes de escribirlos en nuestras páginas web. Para ello debe usar las funciones que provee PHP (o implementar las suyas propias):

- `htmlspecialchars` o `htmlspecialchars` para codificar los caracteres que pueden ser confundidos con otros con significado propio en HTML.
- `strip_tags` para eliminar de la cadena cualquier subcadena que se corresponda con tags HTML.

Resuelva el problema de inyección del ejercicio.

10. Tres en raya

► Ejercicio 14: 3 en raya

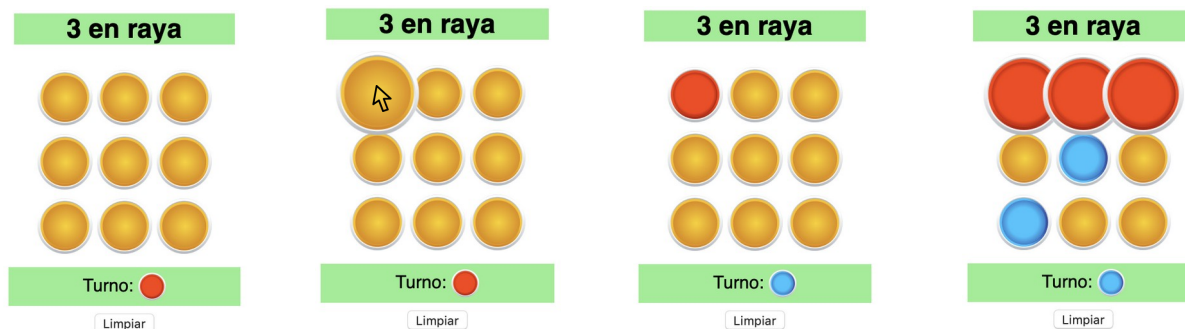
En este ejercicio se pide que implemente el juego del tres en raya con PHP. Desde luego PHP y la comunicación HTTP con un servidor no son la forma más eficiente de hacerlo ya que la interfaz de usuario está en el navegador del cliente y el servidor se necesita para ejecutar la lógica del juego y para generar la interfaz de usuario. Esto obliga a que cada vez que el usuario realiza una acción (pulsar una casilla para poner una ficha), el cliente envía la información de la pulsación al servidor y este genera la página resultante con la ficha puesta y la devuelve al cliente. Sería más adecuado usar *JavaScript*, que permite gestionar mejor la interacción con el usuario y evita la comunicación continua con el servidor: la lógica del juego se ejecutaría en el cliente. Aun así es un ejercicio interesante e incluso divertido. Deberá incluir todo el código necesario en un único fichero .php (HTML, CSS y PHP) ... y no puede usar *JavaScript* en ningún momento.

Una posibilidad es que el tablero sea un formulario con un botón en cada celda del tablero 3x3 (nueve botones), cada uno de los cuales tendrá un atributo "name" diferente. Al pulsar uno de los botones, el formulario será enviado al servidor y este podrá comprobar qué botón ha sido pulsado. Otra posibilidad es usar enlaces en cada celda, de esta forma evitaría el uso de formularios. En este caso, la URL del enlace incluirá un *query string* con información sobre la casilla en la que el usuario desea poner la ficha. Ambas soluciones son similares en cuanto a dificultad y algorítmica.

Tanto el tablero como el turno de juego deben almacenarse en variables de sesión.

La interfaz de usuario debe informar sobre el turno de juego y debe proveer alguna forma de volver a jugar una partida una vez que haya habido empate o haya ganado algún jugador. También deberá informar de alguna forma cuando un jugador gane una partida.

Se muestran algunas capturas de pantalla a modo de ejemplo:



En el ejemplo se utiliza CSS para indicar al usuario que una casilla está disponible (al pasar el

ratón por encima se agranda) o que un jugador ha conseguido tres en raya (se agrandan las casillas alineadas). Esto se hace simplemente indicando que las imágenes son de una clase específica que incluye algunas propiedades que hacen un efecto de zoom. Consulte información sobre las propiedades transform, transition y animation.

En <https://void.ugr.es/practica/3enraya.php> puede probar un *script* basado en el siguiente código HTML+CSS:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>3 en raya</title>
  <style>
    body {
      font-family: helvetica;
    }
    .juego {
      width: 200px;
      margin: auto;
      display: flex;
      flex-direction: column;
      align-items: center;
    }
    .juego > h1 {
      width: 100%;
      background-color: lightgreen;
      text-align: center;
    }
    .juego > .informacion {
      width: 100%;
      background-color: lightgreen;
    }

    .informacion {
      margin: 5px 0;
      padding: 5px;
    }
    .informacion img {
      vertical-align: middle;
    }
    .informacion p {
      text-align: center;
      margin: auto;
    }
    .libre {
      transition: transform .5s ease-in-out;
    }
    .libre:hover {
      transform: scale(1.5);
      cursor: pointer;
    }
    .ganador {
      animation: anim 0.5s infinite linear alternate;
    }
    @keyframes anim {
      from {transform: scale(1);}
      to {transform: scale(1.5);}
    }
  </style>
```

```

</head>
<body>
  <section class="juego">
    <h1>3 en raya</h1>
    <section class="tablero">
      <form method="post" action="">
        <table>
          <tr><td><input type="image" class="libre" src="bamarillo.png" width="50px"
            formmethod="post" name="poner00"/></td>
            <td><input type="image" class="libre" src="bamarillo.png" width="50px"
            formmethod="post" name="poner01"/></td>
            <td><input type="image" src="bazul.png" width="50px" formmethod="post"
            name="poner02"/></td></tr>
          <tr><td><input type="image" src="brojo.png" width="50px" formmethod="post"
            name="poner10"/></td>
            <td><input type="image" src="brojo.png" width="50px" formmethod="post"
            name="poner11"/></td>
            <td><input type="image" class="libre" src="bamarillo.png" width="50px"
            formmethod="post" name="poner12"/></td></tr>
          <tr><td><input type="image" class="libre" src="bamarillo.png" width="50px"
            formmethod="post" name="poner20"/></td>
            <td><input type="image" src="bazul.png" width="50px" formmethod="post"
            name="poner21"/></td>
            <td><input type="image" class="libre" src="bamarillo.png" width="50px"
            formmethod="post" name="poner22"/></td></tr>
        </table>
      </form>
    </section>
    <section class="informacion">
      <p>Turno: </p>
    </section>
    <section class="botones">
      <form method="post" action="">
        <input type="submit" name="limpiar" value="Limpiar"/>
      </form>
    </section>
  </section>
</body>
</html>

```

Sesión 3

Acceso a Bases de Datos

En esta última parte de abordan ejercicios básicos para acceder a bases de datos desde las aplicaciones web.

11. Instalación y configuración de un servidor MySQL

Un gran porcentaje de aplicaciones web almacenan la información con la que trabajan en bases de datos. En esos casos es necesario disponer de un servidor de BBDD, que puede estar en la misma máquina que el servidor web o en una máquina distinta. En esta asignatura se va a usar MySQL, un DBMS de código abierto muy popular que es desarrollado en la actualidad por ORACLE.

La instalación en Ubuntu se realiza con el comando:

```
apt-get install mysql-server
```

Además, la instalación del servidor implica la instalación de `mysql-client`, que incluye un conjunto de programas para acceder al servidor MySQL desde la línea de órdenes. La configuración de MySQL se encuentra en `/etc/mysql/my.cnf`.

Dependiendo de la versión y distribución que esté usando el proceso que continúa puede variar. Cabe la posibilidad de que durante el proceso de instalación se pregunte cual es la clave del administrador del DBMS. En caso de que por algún motivo no tenga la clave de administrador (olvido, etc.) y necesite restaurarla, puede consultar la documentación de MySQL³.

En el aula de prácticas, tras la instalación de `mysql-server`, deberá:

- Ejecutar *script* de configuración. Deberá indicar la clave de *root* y contestar algunas preguntas (la respuesta puede ser 'Sí' en todas ellas):

```
mysql_secure_installation
```

- Cambiar el módulo de autenticación por defecto para el usuario *root* de mysql:
 - Acceder a mysql desde línea de órdenes
`mysql`
 - Cambiar el módulo de autenticación e indicar la clave de *root*:
`ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'ClaveQueQuieraParaElUsuarioRootDeMySQL';`
 - Hacer que los cambios tengan efecto:
`FLUSH PRIVILEGES;`
 - Salir de mysql:
`exit`

Información más detallada aquí:

- Ubuntu 18.04: <https://www.digitalocean.com/community/tutorials/how-to-install-mysql-on-ubuntu-18-04>.
- Ubuntu 20.04: <https://www.digitalocean.com/community/tutorials/how-to-install-mysql-on-ubuntu-20-04>.
- Ubuntu 22.04: <https://www.digitalocean.com/community/tutorials/how-to-install-mysql-on-ubuntu-22-04>.

Una vez instalado, la administración del servidor de BBDD se puede hacer de varias formas:

- Desde la línea de órdenes con el cliente `mysql`.
- Con alguna aplicación que tenga interfaz gráfica.
- Con alguna aplicación web.

11.1. Instalación de una aplicación para administrar la BBDD

Aunque el DBMS se puede administrar desde línea de órdenes, es habitual disponer de alguna aplicación para realizar las tareas de forma más cómoda. Para ello hay dos posibilidades:

- Disponer de una aplicación de escritorio que se ejecute en su máquina local y que se conecte a un servidor remoto/local para su administración.
- Disponer de una aplicación web en el servidor de BBDD para administrarlo.

3 <https://dev.mysql.com/doc/refman/8.0/en/resetting-permissions.html>

Generalmente los servidores de BBDD suelen restringir bastante los accesos por lo que es habitual que solo permitan realizar conexiones desde aquellas máquinas que tienen aplicaciones web que hacen uso de ellos. La conexión suele estar prohibida para el resto de máquinas, es decir, el resto de la red o internet. Por ello, tendrá sentido usar un cliente de escritorio solo cuando ese cliente tenga permiso para la conexión al servidor o bien si el servidor está en la misma máquina que el cliente. Como estas condiciones no son las habituales, es muy frecuente que los servidores de BBDD tengan también instalada una aplicación web para su administración disponible en el mismo servidor de BBDD de forma que podamos conectarnos a través de un navegador web.

11.1.1. Clientes de escritorio

Hay multitud, tanto privativos como de código abierto. Por indicar alguno, puede probar alguno de estos:

- DBeaver⁴. Es un cliente de código abierto multiplataforma de propósito general por lo que puede administrar tanto MySQL como otros SGBD (PostgreSQL, SQLite, Oracle, etc.). Es muy completo.
- MySQL Workbench⁵. Este es un cliente desarrollado específicamente para MySQL por Oracle gratuito muy completo. Suele estar disponible en los repositorios habituales de GNU/Linux por lo que la instalación es muy simple.

Esta opción es adecuada para, por ejemplo, hacer las prácticas de la asignatura en su máquina local. Los clientes de escritorio son muy completos y le permitirán hacer el diseño E-R y crear la BBDD completa de una forma cómoda así como manipular las tablas: visualizar su contenido, insertar o borrar tuplas, etc. Para clonar su BBDD local en **void.ugr.es**, deberá exportarla e importarla desde la aplicación web de administración de BBDD instalada en **void.ugr.es**.

11.1.2. Clientes web

También disponemos de gran variedad de clientes web para administrar BBDD. En caso de que decida usar esta opción (en lugar de instalar una aplicación de escritorio) se recomienda usar phpMyAdmin⁶ que posiblemente es el más popular. Es una aplicación escrita en PHP de código abierto muy completa. Para poder usarla en su máquina local deberá haber instalado y configurado previamente un servidor web (Apache en nuestro caso) y tener habilitada la ejecución de scripts PHP.

En void.ugr.es no se ha instalado phpMyAdmin por cuestiones de seguridad ya que requiere hacer uso de algunas funciones que pueden poner en peligro la privacidad de las prácticas alojadas. Para la administración del servidor MySQL que hay en void.ugr.es se ha instalado la aplicación AdMiner⁷. También es una aplicación escrita en PHP aunque muchísimo más simple que phpMyAdmin, de hecho consta de un único fichero .php. La funcionalidad es bastante básica pero es suficiente para crear tablas, explorarlas o importar una BBDD.

12. Creación de una BBDD

Una vez instalado el DBMS, deberá acceder a él con algún programa administrador de su elección (phpMyAdmin, MySQL-workbench, ...) y crear una base de datos y un usuario nuevo. Si lo hace en su máquina local (localhost) podrá elegir los nombres que desee tanto para la BBDD como para el usuario pero recuerde que la entrega de la práctica se hará en void.ugr.es y que en ese servidor no tiene privilegios para crear BBDD o usuarios nuevos.

Por tanto, para asegurar que cuando suba la práctica a **void.ugr.es** todo funciona sin necesidad de hacer modificaciones en su código, se recomienda que en su localhost cree una BBDD con idénticas características que la que tiene disponible en ese servidor:

- Nombre de la BBDD: USUARIOVOID (el mismo que utiliza en **void.ugr.es**).
- Usuario de la BBDD con privilegios de acceso a la BBDD: USUARIOVOID (el mismo usuario

4 <https://dbeaver.io/>

5 <https://www.mysql.com/products/workbench/>

6 <https://www.phpmyadmin.net>

7 <https://www.adminer.org/>

que utiliza para acceder a `void.ugr.es`).

- Clave de acceso del usuario: la misma que utiliza para acceder a `void.ugr.es`.

Una vez creada la BBDD en su máquina, ya puede empezar a trabajar. En este ejercicio debe crear una pequeña aplicación PHP para gestionar una base de datos de usuarios. Para ello lo primero que hará será crear una tabla llamada usuarios con los campos necesarios para almacenar la información obtenida a partir de los datos del formulario de registro de clientes de hotel. Puede omitir la parte de la fotografía.

Las operaciones habituales en una aplicación que debe hacer el mantenimiento de una BBDD son las conocidas como CRUD, acrónimo de:

- Create: creación de nuevas tuplas, es decir, inserción de nuevos usuarios en la BBDD.
- Read: obtener tuplas de la BBDD. Pueden ser búsquedas atendiendo a algún criterio o listados completos o parciales.
- Update: modificar datos de algún usuario.
- Delete: borrar usuarios.

12.1. Consultas a la BBDD

► Ejercicio 15: Consultas

Cree un *script* que realice las siguientes tareas:

1. Establecer conexión con la BBDD.
2. Mostrar un formulario para solicitar información de una tupla. Puede adaptar el que ha realizado en ejercicios previos. Cuando se pulse el botón de confirmación el *script* hará una inserción (**INSERT**) en la BBDD.
3. Después del formulario, el *script* mostrará siempre un listado de todas las tuplas contenidas en la tabla usuarios (**SELECT**).
4. Cerrar conexión con la BBDD.

Para que este ejercicio funcione deberá haber creado la BBDD y la tabla previamente desde alguna aplicación de administración de la BBDD. Sin embargo, es frecuente que esto se haga desde nuestra propia aplicación. Concretamente, al establecer la conexión con la BBDD puede verificar si existe tanto la BBDD como la tabla de usuarios y, en caso de que no sea así, puede ejecutar las sentencias **CREATE DATABASE**, **GRANT**, **CREATE TABLE**, etc.

Recuerde que las claves (u otra información sensible) se deben almacenar cifradas en la BBDD. Para ello utilice las funciones de PHP:

- `password_verify`
- `password_hash`

Puede ver un ejemplo de uso en <https://www.etutorialspoint.com/index.php/185-php-secure-password-hash-and-password-verify>

Aparte del *script* que implementa la funcionalidad descrita, deberá disponer de algunos ficheros adicionales:

- Un fichero que contenga las credenciales de acceso a la BBDD (nombre de la BBDD, usuario, clave etc.). Este podrá ser código PHP en el que se definan variables con esos valores, JSON, etc.
- Un fichero con funciones u objetos/métodos para establecer la conexión a la BBDD. Este incluirá o cargará el que contiene las credenciales de acceso y será incluido por los *scripts* de la aplicación que estamos desarrollando.
- Un fichero con funciones u objetos/métodos que aporten la funcionalidad requerida por nuestra aplicación en relación con la BBDD, es decir, lo que se conoce como modelo de datos. Concretamente dispondrá de funciones o métodos para:

- Insertar un usuario. A modo de ejemplo, esta podría ser la API:
`insertar_usuario($nombre,$email,$password,$edad,...)`
- Obtener listado de usuarios. A modo de ejemplo, esta podría ser la API:
`obtener_usuarios()`

La función devuelve un *array* con todas las tuplas de la tabla de usuarios.

De esta forma, nuestra aplicación podrá llamar a estas funciones para manipular la BBDD y podemos desacoplar nuestro código PHP del código de acceso (`SELECT`, `INSERT`, `DELETE`, etc.) a la BBDD.

Debe asegurarse de que su código es seguro frente a inyecciones de código HTML y SQL.

12.2. Borrado de tuplas

Mejore el *script* del ejercicio anterior para permitir al usuario que borre tuplas. Para ello, debe añadir un formulario al lado de cada tupla del listado con un botón para borrar. Tras pulsarlo e invocar de nuevo al *script*, este ejecutará un `DELETE` sobre la BBDD para esa tupla y mostrará de nuevo el formulario para añadir tuplas seguido del listado de lo que contiene la BBDD.

Observe que para poder identificar correctamente la tupla que se desea borrar, deberá incluir en el formulario su clave primaria (campo `id`) en un control oculto.

Deberá añadir una función/método al modelo de datos con una API similar a esta:

```
borrar_usuario($id)
```

Para implementar esta funcionalidad puede crear nuevos *scripts* o puede reutilizar un mismo *script*. Es decir, su aplicación puede consistir en un único *script* para realizar todas las operaciones (puede usar el *query string* para distinguir la operación que desea realizar) o puede consistir en un *script* para mostrar el formulario de entrada, otro para borrar tuplas, otro para editar tuplas, etc. Se recomienda usar un único *script* (por ejemplo `index.php`) aunque puede disponer de múltiples ficheros PHP que faciliten la modularización de su código. Estos otros ficheros PHP no están pensados para que un cliente los solicite mediante una URL sino para que sean incluidos desde su *script* `index.php`.

13. Edición de usuarios

Añada un segundo botón a cada tupla que permita editar los datos de ese usuario. La edición consiste en mostrar el formulario de entrada de datos con los controles de entrada rellenos a partir de la información recuperada de la base de datos. Al pulsar el botón para enviar el formulario en lugar de hacer una operación `INSERT` hará una operación `UPDATE`.

Deberá añadir una función/método al modelo de datos con una API similar a esta:

```
modificar_usuario($id,$nombre,$email,$password,$edad,...)
```

14. Listado paginado

Conforme aumenta el tamaño de la BBDD los listados pueden ser bastante extensos. Lo habitual es disponer de una barra de navegación para avanzar mostrar listados parciales avanzando o retrocediendo de N en N. Incluya esta funcionalidad en su *script*.

Deberá modificar la API de la función que devuelve un listado de tuplas para que se adapte a la nueva funcionalidad. Puede añadir parámetros opcionales indicando el rango de tuplas que se desea obtener.

Para hacerlo más completo incluya un formulario para indicar el valor de N. Puede almacenar este dato en variables de sesión o en *cookies*.

15. Entrega de la práctica

El profesor dará instrucciones adicionales sobre lo que hay que incluir en la entrega de esta práctica así como los plazos para ello. En cualquier caso, el material deberá subirse al servidor web de la asignatura (void.ugr.es) y se evaluará únicamente el material subido al servidor.

Para esta práctica en concreto:

- Dentro de su carpeta **public_html** debe incluir una carpeta llamada "**practicaPHP**".
- Dentro de esa carpeta debe incluir todos los ficheros requeridos. Puede usar subdirectorios si lo estima oportuno.
- Debe existir un fichero llamado **index.html** con enlaces, ordenados, a todos y cada uno de los ejercicios realizados.

De esta forma, para acceder al sitio web que ha creado se usará la URL:

<https://void.ugr.es/~USUARIO/practicaPHP>

Observaciones:

- No cumplir con alguno de los requisitos de entrega invalidará la entrega completa.
- No se admitirán entregas por ningún otro medio.

Recomendación: no deje para el último momento la subida de la práctica a **void.ugr.es**.